当前开发环境说明

你的 Windows + VS Code + WSL 仿真工作台

核对日期：2026-11-09。本文依据这台电脑实际安装的工具、项目文件和 VS Code 任务配置编写；版本是核对时的快照，以后自动更新可能改变它们。

你现在的开发方式是：在 Windows 上打开 VS Code 编辑代码，通过 WSL2 的 Ubuntu 运行 Python、编译与仿真 Verilog、打开波形，并用 Chisel 生成 RTL。

每一层分别做什么

层次	你当前使用的东西	作用
电脑系统	Windows 11 家庭中文版，64 位	桌面、文件管理、运行 VS Code
编辑入口	Visual Studio Code	看代码、改代码、选择任务、查看终端输出
Linux 运行环境	WSL2，发行版名 Ubuntu-24.04	在 Windows 内运行 Linux 工具
仿真器	Icarus Verilog / Verilator	在电脑中执行 RTL 的仿真
波形查看器	GTKWave，经 WSLg 显示窗口	打开 VCD，观察时钟、输入和输出
硬件生成工具	Scala CLI + Chisel + Java 21	将 Chisel 描述生成 SystemVerilog

从点击任务到得到结果

```TEXT
Windows：VS Code 中打开 D:\Projects\毕设
            |
            v
选择“运行任务”，或按 Ctrl+Shift+B
            |
            v
Ubuntu-24.04：运行项目的 Python 启动脚本
            |
            +--> Icarus 仿真 --> PASS / 错误 / VCD
            +--> GTKWave     --> 波形窗口
            +--> Scala CLI   --> Chisel 生成的 RTL 文本
```

本周的工作台已经能进行模块级 RTL 仿真与 Chisel 入门实验。Chipyard 的完整处理器系统工程、FPGA 综合实现与板卡验证，是后续另行搭建的阶段。

1. 与你以前的 Vivado 操作怎样对应

原来的工作分给几个小工具完成

以前在 Vivado 中做什么	现在在哪里做	你应该关注什么
写模块文件	VS Code 中编辑 .sv / .scala	接口、位宽、逻辑、寄存器
写仿真文件	VS Code 中编辑 *_tb.sv	时钟、复位、输入变化与检查
点击运行仿真	VS Code 任务调用 Icarus	是否报错，是否出现 PASS
查看波形	GTKWave	每个时间点的输入输出
写引脚和时钟约束	当前小模块仿真暂不使用 XDC	上板阶段再按器件准备
综合、布局布线、下载板卡	尚未配置为本项目工作流	后续根据导师平台决定

testbench 就是你熟悉的“仿真文件”。例如 counter4.sv 是计数器设计，counter4_tb.sv 给它时钟、复位和 enable，并检查 count。仿真文件本身也用硬件描述语言编写，但角色不同。

两条常用路径

```text
手写 Verilog / SystemVerilog
       + 仿真文件
            |
          Icarus
            |
    终端结果 + VCD 波形 --> GTKWave
```

```text
Chisel 源码 + 固定版本配置
            |
   Scala CLI / Scala / Java / firtool
            |
      生成 SystemVerilog RTL
            |
 提取需要的主模块 + 适配器 + 仿真文件
            |
         Icarus 仿真
```

Chisel 的“生成成功”与仿真的“测试通过”是两项检查。当前生成任务将 RTL 打印到终端；检查已有 Chisel 模块的仿真任务读取仓库附带的 .generated.sv，二者没有自动覆盖关系。

2. 当前工具与版本

Windows 上的部分

工具	核对到的版本 / 位置	在本项目中怎么用
Windows	版本 10.0.22631，Windows 11	宿主操作系统
VS Code	1.145.2，x64	编辑和任务入口
WSL 扩展	ms-vscode-remote.remote-wsl 1.104.7	VS Code 连接 Ubuntu
Verilog 扩展	mshr-h.veriloghdl 1.29.0	语法高亮等编辑辅助
Windows Python	3.12.6，通过 py -3 运行	可运行原生 Python 示例

VS Code 的命令入口为 D:\VSC\Microsoft VS Code\bin\code.cmd。WSL 命令入口是 Windows 的 wsl.exe。它们与 Linux 中的同名路径不要混用。

Ubuntu 中的部分

发行版在 WSL 中的名字为 Ubuntu-24.04；系统报告 Ubuntu 24.04.5 LTS；Linux 内核为 6.18.40.1-microsoft-standard-WSL2。当前 Linux 用户为 asus，默认 shell 为 Bash。

工具	核对到的版本	可执行文件位置
Python	3.12.3	/usr/bin/python3
Icarus Verilog	12.0	/usr/bin/iverilog 与 /usr/bin/vvp
GTKWave	3.3.116	/usr/bin/gtkwave
Verilator	5.020	/usr/bin/verilator
Java	OpenJDK 21.0.12.1	/usr/lib/jvm/java-21-openjdk-amd64
Scala CLI	1.9.1	/usr/local/bin/scala-cli
C/C++、Make	GCC 13.3.0、GNU Make 4.3	/usr/bin 下的 gcc、g++、make

Windows 的 Python 与 Ubuntu 的 Python 是两套解释器。当前 VS Code 仿真任务调用 Ubuntu 的 python3。Scala CLI 的 JAR 安装在 /opt/scala-cli/scala-cli.jar，启动脚本不依赖项目的 tmp 目录。

3. Chisel 的依赖为什么还有几个版本号

工具版本与项目版本分开看

Scala CLI 是启动和构建工具；Scala 是编写生成程序的语言；Chisel 是硬件库及相关编译插件；Java 提供运行环境；firtool 把硬件中间表示进一步处理成 RTL。

项目依赖	本项目固定值	从哪里确定
Scala	2.13.16	chisel_project.scala
Chisel	7.2.0	chisel_project.scala
Chisel 编译器插件	7.2.0	chisel_project.scala
Java	当前任务优先使用已有 Java 21	vscode_run.py
firtool	附带生成结果标注 1.133.0	.generated.sv 文件头及生成流程

执行 scala-cli version 会显示它自带或默认选择的 Scala 版本，本机为 3.7.3。这不表示项目改用了 Scala 3；项目的 using scala 指令将实际构建版本固定为 2.13.16。

本机任务怎样选 Java 和依赖仓库

vscode_run.py 先查找已有 Java 21，再把 JDK 路径交给 Scala CLI，可避免每次依赖自动 JDK 选择。若没有可用的 Java 21，启动器才使用 --jvm 21，让 Scala CLI 管理下载。

任务会在该次子进程中将默认依赖仓库设为 Maven Central；如果你已经设置 COURSIER_REPOSITORIES，则保留你的值。没有重写全局 Maven 或 Java 配置。

示例文件的角色

文件	作用与修改方式
SafetyGate.scala	安全门源码；改后重新生成
SyntaxBasics.scala	语法综合源码；对照 Verilog 学习
chisel_project.scala	依赖与编译配置；先保持版本配套
*.generated.sv	已生成的主模块；改设计优先改源码
*_adapter.sv	对接测试平台；随接口变化调整

4. 项目文件到底存在哪里

Windows 与 Ubuntu 看到的是同一批文件

内容	Windows 中的位置	WSL 中的位置
项目根目录	D:\Projects\毕设	/mnt/d/Projects/毕设
学习资料	根目录下的学习资料	同一目录的 Linux 视图
配套代码	学习资料\配套示例	学习资料/配套示例
VS Code 任务	根目录\.vscode\tasks.json	根目录/.vscode/tasks.json
仿真输出	配套示例\.outputs	配套示例/.outputs

这里没有为了 WSL 再复制一份代码。你在 VS Code 改动并保存的文件，就是仿真器接下来读取的文件。Windows 路径通常用反斜杠，Linux 用正斜杠；路径有空格或中文时保留引号。

目录结构

```text
D:\Projects\毕设
  .vscode/                    任务与扩展推荐
  题目/                       题目图片
  会议记录/                   会议纪要
  学习资料/
    当前开发环境说明.md / .pdf
    可编辑源稿/               教材正文与 PDF 制作脚本
    配套示例/
      Windows/、macOS/、Ubuntu-WSL/
      *.sv、*.scala、*.py   源代码与运行脚本
      .outputs/             编译结果、波形和日志
```

运行后生成什么

counter4.vcd 是计数器波形；counter4.gtkw 保存信号列表；gtkwave.log 记录查看器输出；case1.vvp 等是 Icarus 的编译结果。它们在 .outputs 中，由 Git 忽略，不属于需要手工编辑的设计源码。

项目 tmp 目录保存制作资料和测试时的中间文件，也被 Git 忽略。教学工具已经安装在 Ubuntu 的正常位置，使用仿真不需要进入 tmp。

5. 每天打开项目与运行计数器

第一步：打开整个仓库

在 VS Code 使用“文件 → 打开文件夹”，选择 D:\Projects\毕设。不要只打开某个 .sv 文件，否则 VS Code 可能读不到根目录的任务菜单。

有两种窗口模式都能使用当前任务：Windows 窗口下，任务自动调用 wsl.exe 进入 Ubuntu-24.04；WSL 窗口下，任务直接在 Ubuntu 中运行 python3。

第二步：打开设计和仿真文件

```text
学习资料/配套示例/counter4.sv
学习资料/配套示例/counter4_tb.sv
```

前者描述计数器，后者是你熟悉的仿真文件。修改后先保存，再重新运行任务。

第三步：编译并执行仿真

按 Ctrl+Shift+B，执行默认任务“仿真：计数器（第 2 天）”。终端最后应看到：

```text
PASS counter4: reset/hold/increment/wrap
PASS: counter4
```

PASS 表示示例检查的复位、保持、递增和回绕行为通过；仍要结合代码理解它具体检查了什么。

第四步：打开波形

顶部菜单“终端 → 运行任务”，选择“波形：打开计数器波形”。该任务会先重新仿真，再打开 GTKWave，并预置 clk、reset、enable、count 四个信号和完整时间范围。

观察 reset 生效时 count 清零，enable 为 1 时逐拍递增，enable 为 0 时保持，15 再加 1 回到 0。GTKWave 是单独窗口，经 WSLg 显示在 Windows 桌面上。

> 这里的入口是任务菜单与 Ctrl+Shift+B。右上角通用的“运行文件”按钮通常不知道该把哪两个 RTL 文件一起编译，也不知道选哪个仿真顶层。

6. 已配置的任务与运行顺序

在“终端 → 运行任务”中选择

学到哪里	任务名称	运行后看什么
第 1 天，组合逻辑	仿真：选择器（第 1 天）	8 种输入组合的检查结果
第 2 天，寄存器	仿真：计数器（第 2 天）	PASS 与计数序列
看时序变化	波形：打开计数器波形	clk、reset、enable、count
第 2 天，状态机	仿真：安全停止门（第 2 天）	停止状态与放行规则
第 3 天，Chisel	Chisel：生成安全门 RTL（第 3 天）	生成的 module SafetyGate
第 4-5 天	Python：进程与策略示例（第 4-5 天）	创建、管道、终止、规则断言

新增语法与总检查任务

任务名称	含义	对应资料
语法：Verilog 综合示例	仿真手写语法练习电路	01 册 V6
语法：Chisel 综合示例生成 RTL	生成 SyntaxBasics 的 RTL	02 册 C6
语法：仿真已有 Chisel 综合示例	检查已附带的生成版本	02 册 C6
检查：全部 RTL 示例	运行六组 RTL 检查	回归检查已有示例
检查：仿真环境版本	打印工具与 Java 位置	核对环境

“语法综合示例”中的综合是“综合运用语法”的意思，该任务执行功能仿真，没有调用 FPGA 综合实现。生成任务同样不会生成可直接下载到 FPGA 的 bitstream。

需要调整任务时看哪里

所有按钮的实际命令都写在 .vscode/tasks.json。Windows 分支固定使用 Ubuntu-24.04，并将当前项目目录传给 WSL；之后调用配套示例/vscode_run.py，再由它调用 run_examples.py 或波形工具。

如果以后移动项目目录，任务中的工作区变量会跟着变化；如果重新安装 WSL 并改变发行版名称，则需要核对 tasks.json 中的 Ubuntu-24.04。

7. 不点菜单时，怎样手动运行

Windows PowerShell：先打开项目

```POWERSHELL
Set-Location -LiteralPath 'D:\Projects\毕设'
code .
```

这只负责打开编辑器，不执行仿真。在这个 PowerShell 中，手动进入 Linux 可执行：

```POWERSHELL
wsl -d Ubuntu-24.04
```

进入 Ubuntu 后：定位与检查

```BASH
cd '/mnt/d/Projects/毕设'
python3 学习资料/配套示例/vscode_run.py check
```

然后按需选一行运行，不必一次全执行：

```BASH
python3 学习资料/配套示例/vscode_run.py counter4
python3 学习资料/配套示例/vscode_run.py wave
python3 学习资料/配套示例/vscode_run.py chisel
python3 学习资料/配套示例/vscode_run.py python
```

直接调用 Icarus 的最小过程

下面在 Ubuntu 的“配套示例”目录运行：

```BASH
cd '/mnt/d/Projects/毕设/学习资料/配套示例'
iverilog -g2012 -s counter4_tb -o counter4.vvp counter4.sv counter4_tb.sv
vvp counter4.vvp
```

第一行工具命令将模块和仿真文件一起编译；-s 指定仿真顶层，-o 指定编译结果；第二行执行结果文件。手工运行时 VCD 出现在当前目录，任务入口则统一放到 .outputs。

用任务菜单就能完成本周学习；手工命令用于理解工具分工和排查问题。PowerShell 里的 D:\ 路径和 Bash 里的 /mnt/d 路径分别使用，不要混写。

8. 当前已经能做什么

本机验证与适用范围

能力	验证情况	验证日期
Icarus RTL 仿真	六组示例重新运行通过	2026-11-09
Python 进程示例	四个示例重新运行通过	2026-11-09
Chisel RTL 生成	安全门重新生成成功	2026-11-09
GTKWave	计数器 VCD 已打开并查看	2026-11-07
Verilator	计数器编译与运行通过	2026-11-07

工具版本在本次重新核对；波形与 Verilator 记录来自最初安装检查。测试通过仅覆盖所列行为，不等于处理器系统安全架构已完成，也不覆盖 FPGA 布线与物理时序。

本项目尚未搭建的部分

Chipyard 的完整 SoC 工程尚未纳入当前工作区；没有通过这套任务配置执行 FPGA 综合、布局布线、生成 bitstream 或下载板卡。你此前说明本机已不再使用 Vivado，本周示例也不依赖它。

Chipyard 后续用于把处理器、总线、外设和自定义模块组织成系统。当前先完成模块仿真、进程基础和技术目标草案，确定系统实验范围后再选择具体平台。

工具在哪里安装，决定命令在哪里运行

在 PowerShell 中找不到 iverilog，并不能说明 Ubuntu 中没装。当前正式仿真工具位于 Ubuntu；在 Windows 窗口选任务时，任务会替你进入正确环境。

同理，Windows 的 py -3 与 Ubuntu 的 python3 不是同一个解释器；Python 示例可分别运行，进程终止的具体退出码按平台解释。

9. 常见问题与维护

先判断问题属于哪一层

现象	优先检查	操作
运行任务列表为空	是否打开整个项目根目录	重新打开 D:\Projects\毕设
PowerShell 找不到 iverilog	当前终端是 Windows 还是 Ubuntu	直接用任务，或进入 WSL
找不到 counter4_tb	是否编译了仿真文件并选对顶层	使用已配好的计数器任务
波形与修改前一样	文件是否保存、是否重新仿真	保存后重跑波形任务
Chisel 出现下载错误	Java、Maven 或后端下载的哪一层	保留完整报错与对应 URL
GTKWave 无窗口或提示显示错误	WSLg 图形环境	用任务入口，它已配置 X11 后端
同名模块重复定义	是否混编了原模块与适配器	使用分别配置的测试任务

VS Code 更新后的 WSL 后端

VS Code 的 Windows 客户端与 WSL 中的 VS Code Server 需要配套。客户端更新后，首次连接 WSL 可能下载对应版本的后端；这不等于 Icarus、Chisel 或你的源码丢失。

若后端下载缓慢，Windows 本地窗口下的现有任务仍会直接调用 wsl.exe，通常无需等待远程编辑后端才能执行仿真。下载完成前，避免把“连接编辑器失败”误判成“RTL 测试失败”。

更新与备份

源码、任务、教材源稿和生成示例由 Git 管理；.outputs、.scala-build、.bsp、tmp 等缓存由 .gitignore 排除。换一台电脑时，需要重新安装工具；克隆仓库只取得文件，不会自动安装 Linux 软件。

VS Code 与扩展可以自动更新，教学依赖暂保持固定版本。升级 Chisel、Scala 或插件前，先保留当前可用配置，再重新运行生成与六组 RTL 检查。

一句话复述给别人

“我在 Windows 上用 VS Code，通过 WSL2 的 Ubuntu 做 Verilog/SystemVerilog 功能仿真；Icarus 负责运行，GTKWave 看波形，Chisel 经 Scala CLI 和 Java 21 生成 RTL，后续再接入处理器系统实验。”